KickApp

Project Plan

Francesco Scaburri Davide Fasolini

Ingegneria del Software – A.A. 2026/27

Indice

1 Introduzione **3**
1.1 Panoramica . . . 3
1.2 Obiettivi . . . 3

2 Modello di processo **4**
2.1 Metodologia Agile/Scrum . . . 4
2.2 Ruoli e responsabilità Scrum . . . 4
2.3 Approccio MDA . . . 4

3 Organizzazione del progetto **4**
3.1 Struttura del team di progetto . . . 4
3.2 Ruoli e interazioni con l'esterno . . . 5
3.3 Strumenti di gestione e collaborazione . . . 5

4 Standard, linee guida e procedure **5**

5 Attività di gestione **5**

6 Rischi **5**
6.1 Disponibilità di tempo limitata . . . 6
6.2 Numero limitato di componenti . . . 6
6.3 Difficoltà nell'utilizzo di nuove tecnologie . . . 6

7 Personale **6**

8 Metodi e tecniche **6**
8.1 Ingegneria dei requisiti . . . 6
8.2 Progettazione e modellazione . . . 6
8.3 Implementazione . . . 7
8.4 Testing . . . 7
8.5 Controllo della versione e della configurazione . . . 7

9 Garanzia di qualità **7**

10 Pacchetti di lavoro (work packages) **7**

11 Risorse **8**

12 Budget **8**

13 Cambiamenti **8**

14 Consegna **8**

1 Introduzione

1.1 Panoramica

KickApp è un software desktop sviluppato in linguaggio Java, progettato per automatizzare, semplificare e ottimizzare la gestione operativa e amministrativa di una palestra di Kickboxing.

L'idea alla base di KickApp nasce dall'analisi delle inefficienze reali che caratterizzano la gestione quotidiana dei corsi di arti marziali e sport da combattimento quando è affidata a strumenti tradizionali come registri cartacei, fogli di calcolo isolati o comunicazioni verbali. In molte realtà sportive tali metodi generano criticità rilevanti su tre fronti principali:

- **Criticità amministrative e legali**: la verifica della regolarità del tesseramento e della validità del certificato medico d'idoneità sportiva avviene spesso in modo manuale e discontinuo. La scadenza inosservata di un certificato o di un abbonamento espone la struttura a gravi responsabilità legali e rischi amministrativi, consentendo ad atleti non in regola di accedere alla sala allenamenti e salire sul ring.
- **Criticità operative e di sicurezza**: nelle discipline da contatto come la Kickboxing, la gestione della capienza massima e del numero di partecipanti per lezione è un fattore critico per la sicurezza degli atleti e per la qualità dell'insegnamento. La mancanza di un sistema di prenotazione in tempo reale porta frequentemente a situazioni di sovraffollamento in sala o, al contrario, all'impossibilità per gli istruttori di pianificare adeguatamente la sessione senza conoscere in anticipo il numero e il livello tecnico degli iscritti.
- **Tracciamento dei progressi dell'atleta**: nel percorso di crescita nelle arti marziali, la costanza e la frequenza agli allenamenti rappresentano il requisito fondamentale per l'ammissione agli esami di passaggio di cintura. Nei sistemi tradizionali l'atleta non dispone di uno strumento immediato e trasparente per monitorare il proprio storico presenze, il conteggio delle lezioni completate o lo stato del proprio avanzamento di grado, con conseguente incertezza e scarsa motivazione.

KickApp risponde a queste problematiche offrendo un'applicazione desktop locale, fruibile con interfacce e permessi dedicati a tre ruoli distinti: Amministratore, Insegnante/Istruttore e Atleta. L'applicazione utilizza un database **embedded** e non richiede l'installazione di servizi esterni, così da poter essere eseguita localmente senza configurazioni aggiuntive.

1.2 Obiettivi

Per rispondere in modo puntuale e affidabile alle criticità individuate, KickApp si pone i seguenti obiettivi fondamentali:

- **Profilazione e controllo degli accessi basato sui ruoli**: fornire un sistema di autenticazione sicuro, con interfacce e permessi differenziati per Amministratore, Insegnante e Atleta.
- **Gestione integrata di abbonamenti e certificati medici**: automatizzare il controllo di validità dei tesseramenti e delle certificazioni sanitarie, bloccando preventivamente le prenotazioni alle lezioni per gli atleti la cui posizione risulti irregolare o scaduta.
- **Programmazione del calendario e gestione della capienza**: consentire agli Insegnanti di programmare le sessioni di allenamento impostando parametri quali orario, disciplina e **capienza massima della sala**, offrendo agli Atleti la possibilità di iscriversi o disdire in tempo reale, con il conteggio dei posti residui.
- **Registro presenze e check-in dell'Istruttore**: garantire la registrazione delle presenze alle lezioni tramite spunta a cura dell'Insegnante al termine della sessione, assicurando che solo le presenze confermate incrementino lo storico dell'allievo.

- **Dashboard "I miei progressi" e tracciamento della cintura**: mettere a disposizione dell'Atleta una sezione dedicata al monitoraggio personalizzato del proprio percorso sportivo, comprensiva del conteggio totale delle lezioni frequentate e dello storico dei passaggi di grado/cintura.

2 Modello di processo

2.1 Metodologia Agile/Scrum

Per lo sviluppo di KickApp viene adottato il framework Scrum, un modello di processo Agile basato sullo sviluppo iterativo e incrementale. Scrum è stato preferito ad altri approcci per la sua flessibilità e per la capacità di adattarsi bene a un team di due persone. Questo approccio permette di strutturare correttamente il lavoro senza sacrificare la capacità di adattamento ai cambiamenti che emergono durante lo sviluppo. Il progetto è organizzato in Sprint della durata di una settimana ciascuno.

2.2 Ruoli e responsabilità Scrum

- **Scrum Master**: ruolo ricoperto a rotazione, a ogni Sprint, tra i due componenti del team; chi lo svolge si assicura che il processo venga eseguito correttamente e che il team resti coeso e concentrato sul lavoro dello Sprint in corso.
- **Product Owner**: ruolo gestito in modalità condivisa tra entrambi i membri del team, per la definizione delle priorità del Product Backlog e la validazione dei requisiti. Qualora necessario, il team si avvarrà del supporto di un proprietario di una palestra di Kickboxing per comprendere meglio il dominio.

2.3 Approccio MDA

Il processo integra parzialmente un approccio MDA (*Model Driven Architecture*): a partire dal Class Diagram realizzato in StarUML viene generato lo scheletro iniziale delle classi Java, che viene successivamente completato e modificato manualmente (ad esempio implementando i corpi dei metodi). Le classi generate a partire dal modello UML verranno indicate nella documentazione finale del progetto.

3 Organizzazione del progetto

Entrambi i componenti del gruppo parteciperanno alla gestione del progetto, con l'ausilio del proprietario della palestra di riferimento nella fase di elicitazione dei requisiti. Tale processo sarà documentato nella Specifica dei Requisiti, che costituisce la base per le attività successive dello sviluppo.

3.1 Struttura del team di progetto

Per assicurare la massima flessibilità e una copertura completa di tutte le attività necessarie non viene adottata una suddivisione rigida dei ruoli; entrambi i componenti partecipano in modo integrato alle seguenti mansioni:

- **Project Manager/Scrum Master**: gestione della pianificazione, coordinamento degli Sprint e monitoraggio delle scadenze.
- **Analista dei requisiti**: elicitazione e formalizzazione dei requisiti.
- **Progettista e modellatore UML**: redazione e gestione dei modelli UML.
- **Sviluppatore software**: scrittura del codice Java in ambiente Eclipse.

- **Tester/Quality Assurance**: scrittura ed esecuzione dei test unitari e verifica dei report di copertura.
- **Responsabile della documentazione**: stesura e revisione continuativa delle relazioni di progetto.

3.2 Ruoli e interazioni con l'esterno

Durante il ciclo di vita del progetto il team si interfaccia con un Esperto di Dominio (Gestore/Maestro di Kickboxing), figura chiave interpellata per identificare i flussi di lavoro reali della palestra (scadenze dei certificati, frequenza alle lezioni, passaggi di grado delle cinture).

3.3 Strumenti di gestione e collaborazione

Il coordinamento interno e il tracciamento delle attività si avvalgono dei seguenti strumenti:

- **Repository GitHub**: controllo di versione per il codice sorgente Java e per i file di documentazione.
- **GitHub Issues e Pull Request**: tracciamento dei singoli compiti e bug e gestione delle revisioni del codice prima dell'integrazione nel branch principale.
- **GitHub Projects (Kanban Board)**: gestione visiva dello stato di avanzamento dei task, suddivisi nelle colonne *To Do*, *In Progress* e *Done*.
- **Discord e WhatsApp**: canali per la comunicazione quotidiana rapida e per lo svolgimento di call periodiche di allineamento.

4 Standard, linee guida e procedure

Lo sviluppo di **KickApp** si conforma a standard metodologici e tecnologici volti a garantire la qualità, l'omogeneità e la manutenibilità dell'applicazione. Il codice sorgente Java viene redatto nel rispetto delle convenzioni ufficiali Oracle e documentato tramite sintassi Javadoc. La Specifica dei Requisiti Software (SRS) viene formalizzata secondo lo standard **IEEE 830**, categorizzando i requisiti con la prioritizzazione **MoSCoW**. La documentazione sarà redatta utilizzando il linguaggio UML, tramite lo strumento StarUML, con l'obiettivo di ottenere diagrammi chiari e rappresentativi del dominio. Lo sviluppo e l'implementazione verranno interamente svolti nell'IDE Eclipse.

5 Attività di gestione

Il coordinamento delle attività operative avverrà attraverso incontri periodici di allineamento, svolti sia di persona sia da remoto tramite la piattaforma Discord, volti a monitorare lo stato dei task e a risolvere tempestivamente eventuali ostacoli tecnici. Alla conclusione di ciascuno Sprint di una settimana verrà effettuata una Sprint Review per verificare le funzionalità sviluppate e pianificare gli obiettivi dell'iterazione successiva. L'assegnazione dei compiti, la gestione delle segnalazioni e le richieste di modifica dei requisiti saranno formalizzate mediante Issue aperte sulla board di progetto in GitHub Projects. Ogni nuovo sviluppo o intervento correttivo verrà eseguito su un branch dedicato e integrato nel ramo principale solo a valle di una Pull Request, con relativa revisione del codice effettuata dal compagno di team.

6 Rischi

6.1 Disponibilità di tempo limitata

La concomitanza con altri esami universitari, lezioni o impegni personali potrebbe ridurre le ore di lavoro effettivamente dedicate al progetto, con il rischio di causare ritardi nella tabella di marcia e di compromettere il completamento di tutti gli obiettivi prefissati.

6.2 Numero limitato di componenti

Il team è composto da soli due componenti; di conseguenza, l'insorgere di problemi tecnici complessi su un modulo o l'eventuale indisponibilità temporanea di uno dei membri graverebbero in modo significativo sul carico di lavoro complessivo e sul prodotto finale.

6.3 Difficoltà nell'utilizzo di nuove tecnologie

L'implementazione dell'applicazione richiede l'impiego di diversi strumenti e framework con cui i componenti del team hanno una familiarità parziale. Ciò potrebbe comportare un maggiore dispendio di tempo dovuto alla curva di apprendimento.

7 Personale

Il progetto viene condotto da un team auto-organizzato composto da **Francesco Scaburri** e **Davide Fasolini**. Secondo i principi del framework Scrum, la gestione del personale non prevede una struttura gerarchica rigida, bensì un'allocazione dinamica e paritaria delle responsabilità. Ciascun membro partecipa attivamente a tutte le fasi del ciclo di vita del software, dall'analisi dei requisiti alla modellazione, fino allo sviluppo in ambiente Eclipse e al testing con JUnit. La figura dello Scrum Master viene riassegnata a rotazione all'inizio di ogni Sprint tra i due componenti.

8 Metodi e tecniche

In tutte le fasi del ciclo di vita del software vengono applicate metodologie e tecniche consolidate, per garantire un processo di sviluppo controllato e conforme ai requisiti d'esame.

8.1 Ingegneria dei requisiti

Per l'analisi e la formalizzazione dei requisiti il team adotta le seguenti tecniche:

- **Etnografia**: i componenti del team partecipano attivamente alle attività della palestra di Kickboxing; hanno quindi cognizione di causa e sanno individuare i bisogni e le difficoltà che gli atleti incontrano quotidianamente.
- **Derivazione da un sistema esistente**: partendo da Maat, un'applicazione già esistente utilizzata da un'altra palestra, che offre funzionalità simili ma per uno sport diverso (e quindi non perfettamente aderenti al dominio), il team può trarre ispirazione per alcuni aspetti (grafica e alcune funzionalità).
- **Prioritizzazione MoSCoW**: categorizzazione di ogni requisito nei quattro livelli di priorità (*Must Have*, *Should Have*, *Could Have*, *Won't Have*) per guidare la pianificazione degli Sprint.

8.2 Progettazione e modellazione

La fase di progettazione si avvale dello standard **UML versione 5** per definire l'architettura del sistema:

- **StarUML**: strumento di riferimento per la realizzazione dei diagrammi architetturali e per la generazione automatica dello scheletro iniziale del codice Java a partire dal Class Diagram.

- **Diagrammi UML prodotti**: copertura dei diagrammi strutturali e comportamentali previsti dalle linee guida del corso (Use Case, Class, State Machine, Sequence, Communication, Activity, Package e Component Diagram).

8.3 Implementazione

L'applicazione sarà sviluppata in linguaggio Java e gestita tramite Eclipse con struttura Maven. Durante l'implementazione verranno adottate le convenzioni di codifica di Java.

8.4 Testing

La correttezza e la qualità del software vengono garantite mediante la progettazione e l'esecuzione di test unitari automatici sviluppati con il framework **JUnit 5**, integrati nella struttura di build **Maven** ed eseguiti all'interno dell'IDE Eclipse. Per assicurare un'adeguata efficacia delle suite di test viene inoltre analizzata la copertura del codice (*test coverage*) per i package e le classi principali del sistema.

8.5 Controllo della versione e della configurazione

Per il versioning del codice sorgente e la gestione delle configurazioni di KickApp, il team adotta Git con repository remoto ospitato su GitHub. Il flusso di lavoro si basa su una strategia a branch dedicati, in cui ogni nuova funzionalità o correzione viene sviluppata in isolamento e integrata nel ramo principale unicamente a fronte di una Pull Request e della conseguente Code Review incrociata svolta dal compagno di team. La tracciabilità delle attività, dei requisiti e delle eventuali segnalazioni è garantita dall'impiego coordinato di GitHub Issues e GitHub Projects (organizzato come board Kanban). Oltre al codice Java, il repository mantiene sotto controllo di versione l'intero patrimonio informativo del progetto, inclusi i file di build di Maven, i modelli UML e la documentazione di accompagnamento.

9 Garanzia di qualità

Per garantire che l'applicazione soddisfi elevati standard di affidabilità, correttezza e manutenibilità, il progetto si ispira ai criteri del modello di **McCall**. La qualità complessiva viene verificata continuamente durante il ciclo di vita del software mediante l'integrazione di procedure organizzative e tecniche:

- l'adozione del framework **Agile/Scrum** con Sprint Review periodiche;
- la formalizzazione chiara di requisiti e architettura secondo gli standard **IEEE 830** e **UML**;
- l'esecuzione automatizzata di test unitari con **JUnit 5** e **Maven**, per prevenire regressioni nel codice;
- la tracciabilità di ogni modifica su **GitHub**, coordinata da Issue e Pull Request con code review.

10 Pacchetti di lavoro (work packages)

Il progetto viene realizzato suddividendo il lavoro nei seguenti pacchetti:

- Raccolta e analisi dei requisiti
- Elaborazione della documentazione
- Progettazione e modellazione UML
- Implementazione del software

- Gestione dei dati
- Testing e verifica

11 Risorse

Durante lo sviluppo dell'applicazione verranno impiegati i seguenti strumenti software, necessari per la realizzazione, la comunicazione, la modellazione e la documentazione del sistema:

- **Overleaf**: editor LaTeX utilizzato per la stesura collaborativa e la formattazione della documentazione e del Project Plan.
- **Discord**: piattaforma impiegata per le riunioni a distanza, i briefing di allineamento e il coordinamento quotidiano del gruppo.
- **Eclipse**: ambiente di sviluppo integrato (IDE) ufficiale per la scrittura e il debug del codice sorgente in linguaggio Java.
- **Apache Maven**: sistema per la gestione automatizzata delle dipendenze di progetto e del ciclo di build (tramite il file `pom.xml`).
- **StarUML**: strumento adottato per la modellazione dei diagrammi UML e per la generazione dello scheletro delle classi Java.
- **JavaFX**: framework grafico dedicato alla realizzazione dell'interfaccia utente (GUI) dell'applicazione.
- **GitHub/GitHub Projects**: piattaforma per il controllo di versione (Git), il tracciamento delle Issue e la gestione della board Kanban.

12 Budget

Lo sviluppo dell'applicazione non comporta costi finanziari diretti, poiché si avvale esclusivamente di ambienti e strumenti software a licenza gratuita o open source (IDE Eclipse, StarUML, Apache Maven, JUnit 5, GitHub, Discord e Overleaf). Il budget di progetto è espresso unicamente in termini di capacità operativa e tempo di lavoro: prevedendo un impegno stimato di circa 40 ore a persona per ciascuno dei 2 componenti del team, la disponibilità complessiva ammonta a circa **80 ore lavorative totali**.

13 Cambiamenti

Eventuali modifiche o integrazioni ai requisiti e alle funzionalità dell'applicazione verranno gestite in modo sistematico, per evitare impatti negativi sui tempi di consegna. Ogni proposta di variazione dovrà essere motivata e formalizzata tramite la creazione di un'apposita Issue su GitHub. Il team valuterà l'impatto della modifica in termini di fattibilità tecnica e di impegno temporale; se approvata, la modifica comporterà l'aggiornamento della Specifica dei Requisiti e verrà sviluppata su un branch dedicato, garantendo la piena tracciabilità del software.

14 Consegna

La consegna finale di KickApp avverrà tramite la condivisione del repository remoto GitHub con i docenti del corso. Il repository conterrà sia la versione finale e stabile del codice sorgente Java (progetto Eclipse/Maven) sia la documentazione completa prodotta durante il ciclo di vita del software (Project Plan, Specifica dei Requisiti IEEE 830, file del modello StarUML, suite di test JUnit 5 e manuale utente).